

```
pradeepkumar@pradeepkumar ~ $ cd RIOT-master/examples/
pradeepkumar@pradeepkumar ~ $ cd RIOT-master/examples $ ls
default      gpio_networking  ipm_pingpong  riot_end.cpy
gpio_board.cup hello-world  ipm_pingpong  riot_end.cpy
pradeepkumar@pradeepkumar ~ $ cd RIOT-master/examples $ cd hello-world/
pradeepkumar@pradeepkumar ~ $ cd RIOT-master/examples/hello-world $ ls
bin main.c Makefile README.ad
pradeepkumar@pradeepkumar ~ $ cd RIOT-master/examples/hello-world $ make flash
Building application 'hello-world' for 'native' with MCU 'native'.
make* -C /home/pradeepkumar/RIOT-master/boards/native
make* -C /home/pradeepkumar/RIOT-master/boards/native/drivers
make* -C /home/pradeepkumar/RIOT-master/cpu
make* -C /home/pradeepkumar/RIOT-master/cpu/native
make* -C /home/pradeepkumar/RIOT-master/cpu/native/periph
make* -C /home/pradeepkumar/RIOT-master/drivers
make* -C /home/pradeepkumar/RIOT-master/sys
make* -C /home/pradeepkumar/RIOT-master/targets/native
text data bss doc hex filename
24428 192 47636 73448
/home/world/bin/native/hello-world.elf
make
pradeepkumar@pradeepkumar ~ $ cd RIOT-master/examples/hello-world $
```

Figure 3: Running a 'hello-world' example

```
make* -C /home/pradeepkumar/RIOT-master/boards/native
make* -C /home/pradeepkumar/RIOT-master/boards/native/drivers
make* -C /home/pradeepkumar/RIOT-master/cpu
make* -C /home/pradeepkumar/RIOT-master/cpu/native
make* -C /home/pradeepkumar/RIOT-master/cpu/native/periph
make* -C /home/pradeepkumar/RIOT-master/drivers
make* -C /home/pradeepkumar/RIOT-master/sys
make* -C /home/pradeepkumar/RIOT-master/targets/native
text data bss doc hex filename
24428 192 47636 73448
/home/world/bin/native/hello-world.elf
make
pradeepkumar@pradeepkumar ~ $ cd RIOT-master/examples/hello-world $ make term
make* -C /home/pradeepkumar/RIOT-master/examples/hello-world/bin/native/hello-world.elf
RIOT native interrupts/signals initialized.
LED_GREEN_ON
LED_RED_ON
RIOT native board initialized.
RIOT native hardware initialization complete.
main(): This is RIOT! (Version: UNKNOWN (buildir: /home/pradeepkumar/RIOT-master))
RIOT socket example application
All up, running the shell now
> help
udp                send data over UDP and listen on UDP
ports              Reboot the node
reboot             Prints information about running
ps                 threads.
random_init         initializes the PRNG
random_get          returns 32 bit of pseudo randomness;
ifconfig            Configure network interfaces
txtsnd             send raw data
ncache             manage neighbor cache by hand
routers            IPv6 default router list
```

Figure 4: The 'hello-world' application being run in the terminal

```
pradeepkumar@pradeepkumar ~ $ cd RIOT-master/examples/posix_sockets $ sudo make term
/home/pradeepkumar/RIOT-master/examples/posix_sockets/bin/native/posix_sockets.elf tap0
RIOT native interrupts/signals initialized.
LED_GREEN_ON
LED_RED_ON
RIOT native board initialized.
RIOT native hardware initialization complete.
main(): This is RIOT! (Version: UNKNOWN (buildir: /home/pradeepkumar/RIOT-master))
RIOT socket example application
All up, running the shell now
> help
Command            Description
-----
udp                send data over UDP and listen on UDP ports
ports              Reboot the node
reboot             Prints information about running threads.
ps                 initializes the PRNG
random_init        returns 32 bit of pseudo randomness
random_get         Configure network interfaces
ifconfig           send raw data
txtsnd            send raw data
ncache            manage neighbor cache by hand
routers            IPv6 default router list
```

Figure 5: POSIX socket application in RIOTCommand

You are running RIOT on a(n) native board.

This board features a(n) native MCU.

Press Ctl+C to quit the application

`make flash` will compile and show the text and the size of the data occupied by the memory. Since this application is run under native mode, the output can be viewed using the `make term` command (refer Figures 3 and 4).

Networking applications in RIOT

There are a few applications available in RIOT. One such app is a socket application, which works with the POSIX API with UDP as the protocol. All the relevant source code is available in the directory itself.

```
pradeepkumar $ ] cd RIOT-Master/examples/posix-socket/
```

```
pradeepkumar $ ] make flash
```

```
pradeepkumar $ ] sudo make term
```

output of the above command

```
/home/pradeepkumar/RIOT-master/examples/posix_sockets/bin/
native/posix_sockets.elf tap0
```

RIOT native interrupts/signals initialized.

LED_GREEN_OFF

LED_RED_ON

RIOT native board initialized.

RIOT native hardware initialization complete.

main(): This is RIOT! (Version: UNKNOWN (buildir: /home/

pradeepkumar/RIOT-master))

RIOT socket example application

All up, running the shell now

> help

udp send data over UDP and listen on UDP

ports

reboot Reboot the node

ps Prints information about running

threads.

random_init initializes the PRNG

random_get returns 32 bit of pseudo randomness;

ifconfig Configure network interfaces

txtsnd send raw data

ncache manage neighbor cache by hand

routers IPv6 default router list

Figure 5 shows the above configuration; routers can be configured, text messages can be sent and details about running processes can be searched using this application.

There are only a limited number of OSs available for IoT sensors and, among these, RIOT really has a well-documented API with huge support for most of the boards, architecture and sensors. Its main advantages are that it is actively developed and maintained, and there is an incremental version every month. Students and electronics hobbyists/enthusiasts are using Arduino and Raspberry Pi for their IoT applications these days, and RIOT really helps these boards and sensors to achieve a low memory footprint and high energy efficiency without any compromises on the performance of the IoT systems. **END** 

References

- [1] <http://www.riot-os.org/>
- [2] <http://www.riot-os.org/docs/riot-infocom2013-abstract.pdf>
- [3] <http://ieeexplore.ieee.org/xpl/articleDetails.jsp?amumber=5355049>
- [4] <https://hal.inria.fr/hal-00768685/>

By: T.S. Pradeepkumar

The author is a faculty member in the Chennai campus of VT University, where he deploys e-learning software such as Moodle and others, to make the campus paper-free. He can be contacted at pradeepkumarts@gmail.com and through his websites, <http://www.nsnam.com> and <http://www.linuxasaservice.com>